CS4432: Database Systems II

Introduction

Course Staff

- **Roee Shraga**
  - Office: UH 353
  - Office Hours : Mon 2:30PM-3:30PM
    - (contact ahead for zoom availability)
  - Pronouns: he/his/him

Course Staff

- **Abbas Jivan (he/him)**
  - Office: Unity Hall 348
  - Office Hours: Tue 4:00 PM - 5:00 PM
- **Trang Tran (she/her)**
  - Office: Unity Hall 347
  - Office Hours: Mon and Thu, 2:00 PM – 3:00 PM
- **Hemanth Vadlamani (he/him)**
  - Zoom (https://wpi.zoom.us/j/2477868643)
  - Office Hours: Wed 10:00 AM – 11 AM

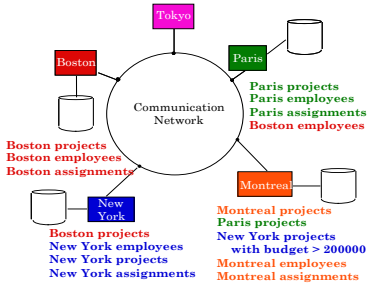
Scope of Data Management

- Changes in DB driven by data
- Current Data: Variety + Volume ➔ ***Big Data***

Relational DBs

MapReduce & Hadoop

Data Stream Management Systems



Distributed DBs

NoSQL: NotOnly SQL

What *is* a Relational Database Management System?

Database Management System = DBMS

Relational DBMS = RDBMS

- **A collection of files that store the data**
  - But: Files that we do not directly access or read
- **A big C program written by someone else that accesses and updates those files for you**
  - But: Huge program containing 100s of 1000s of lines

Where are RDBMS used?

- Backend for traditional “database” applications
- Backend for large Websites
- Backend for Web services

University registration

Bank accounts and transactions

Airline reservation

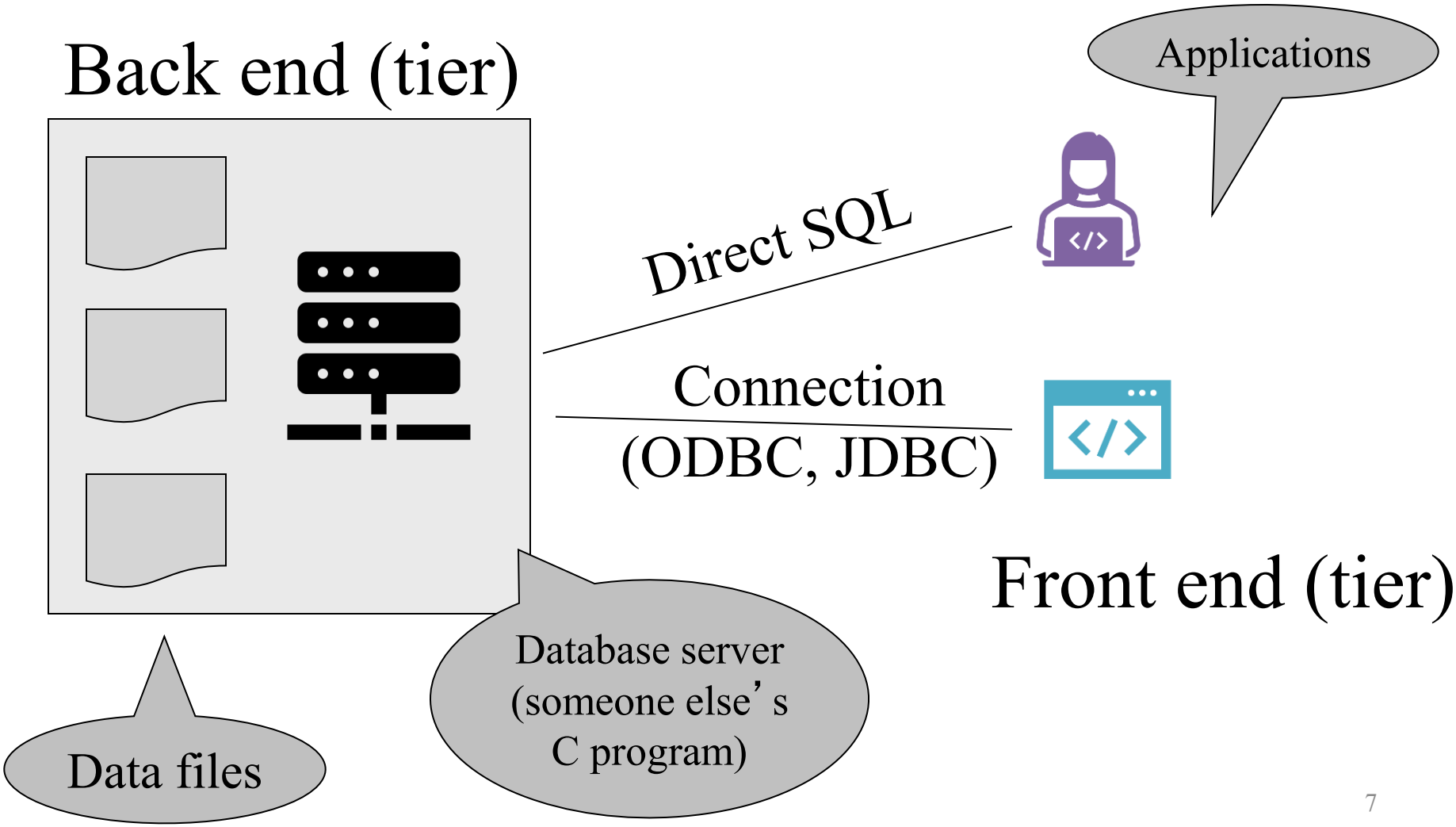
Hospital System

IMDb®

Movie Database

Enters a DBMS

"Two tier database system"



Functionality of a DBMS

The programmer sees SQL, which has two components:

- Data Definition Language - DDL
- Data Manipulation Language – DML

Frontend (CS3431)

Behind the scene the DBMS has:

- Query Optimizer
- Query Engine
- Storage Management
- (Transaction Management, e.g., concurrency, recovery)

Backend (CS4432)

How the Programmer Sees the DBMS

Frontend

- Start with DDL to *create tables*:

```
CREATE TABLE Students (
        Name CHAR(30)
        SSN CHAR(9) PRIMARY KEY NOT NULL,
        Category CHAR(20)
)  . . .
```

- Continue with DML to *populate tables:*

```
INSERT INTO Students
VALUES( 'Charles' , '123456789' , 'undergraduate' )
. . . .
```

WHAT IF YOU DON'T REMEMBER SQL…

How the Programmer Sees the DBMS

Frontend

- Tables:

Students:

SSN	Name	Category
123-45-6789	Charles	undergrad
234-56-7890	Dan	grad
	…	…

Takes:

SSN	CID
123-45-6789	CSE444
123-45-6789	CSE444
234-56-7890	CSE142
	…

Courses:

CID	Name	Quarter
CSE444	Databases	fall
CSE541	Operating systems	winter

"*data independence*" = separate *logical* view from *physical implementation*

What is Hidden ???

Backend

```
CREATE TABLE Students (
        Name CHAR(30)
        SSN CHAR(9) PRIMARY KEY NOT NULL,
        Category CHAR(20)
)  . . .
```

Creating file (Data)

Updating catalog tables

May create indexes

```
INSERT INTO Students
VALUES( 'Charles' , '123456789' , 'undergraduate' )
. . . . .
```

Reading and updating file from disk

Check constraints

May update indexes

Queries

Frontend

- Find all course names that "Mary" takes

```
SELECT  C.name
FROM    Students S, Takes T, Courses C
WHERE   S.name="Mary" and
        S.ssn = T.ssn and T.cid = C.cid
```

- ***We did not specify how to execute***
- ***We did not specify how to optimize***

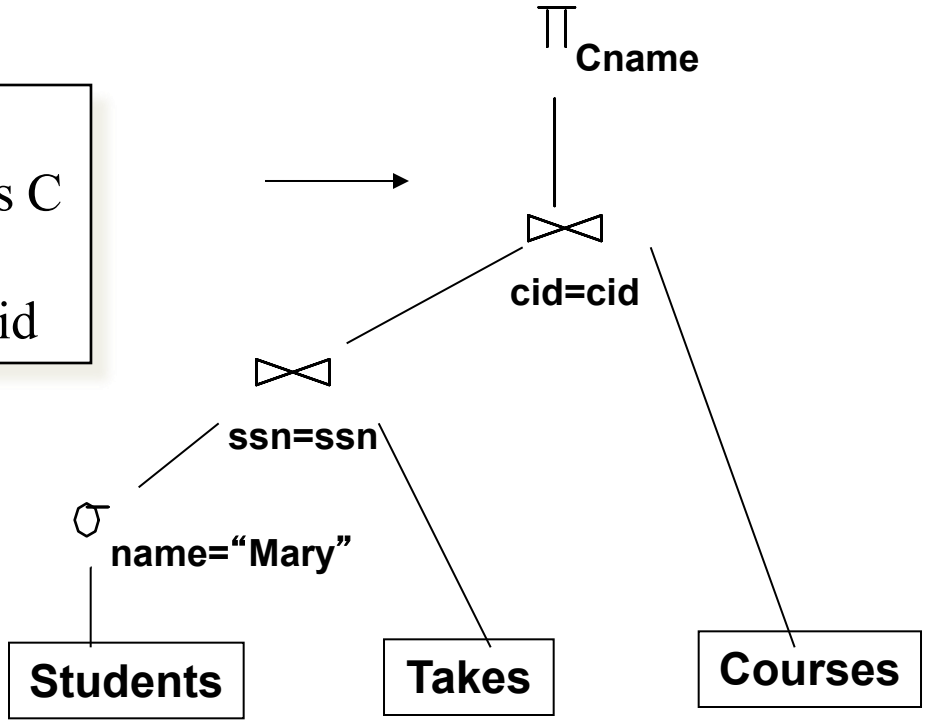
An end-user should be happy not to do it…

What is Hidden ???

Backend

Declarative SQL query → *Imperative query execution plan:*

```
SELECT  C.name
FROM Students S, Takes T, Courses C
WHERE S.name="Mary" and
        S.ssn = T.ssn and T.cid = C.cid
```



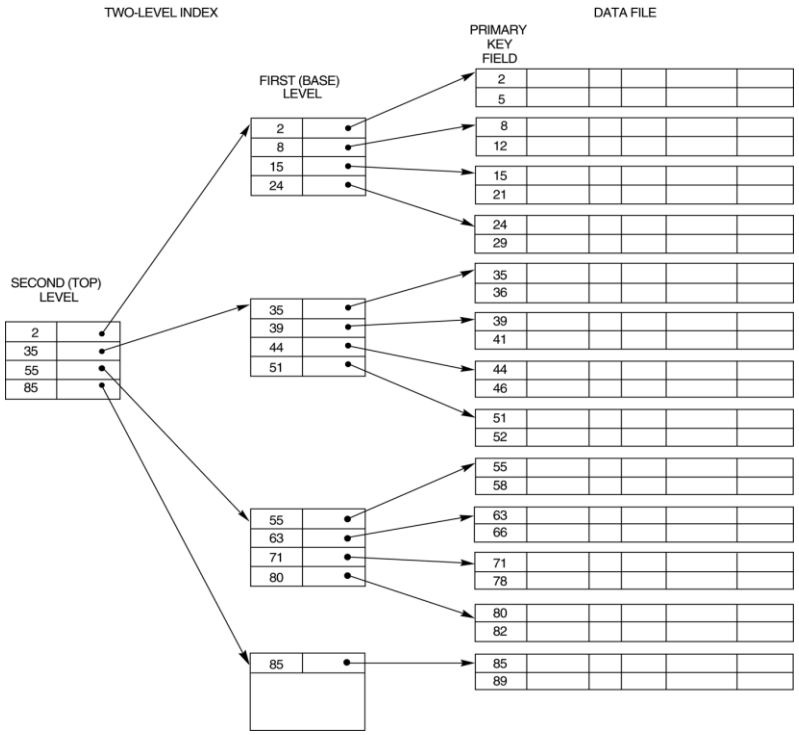
Complex cost model & many alternatives

The optimizer chooses the best execution plan for a query (may involve indexes)

Usage of an Index

```
Select ID, name, address
From Student
Where ID = 1000;
```

Build an index



Millions of records

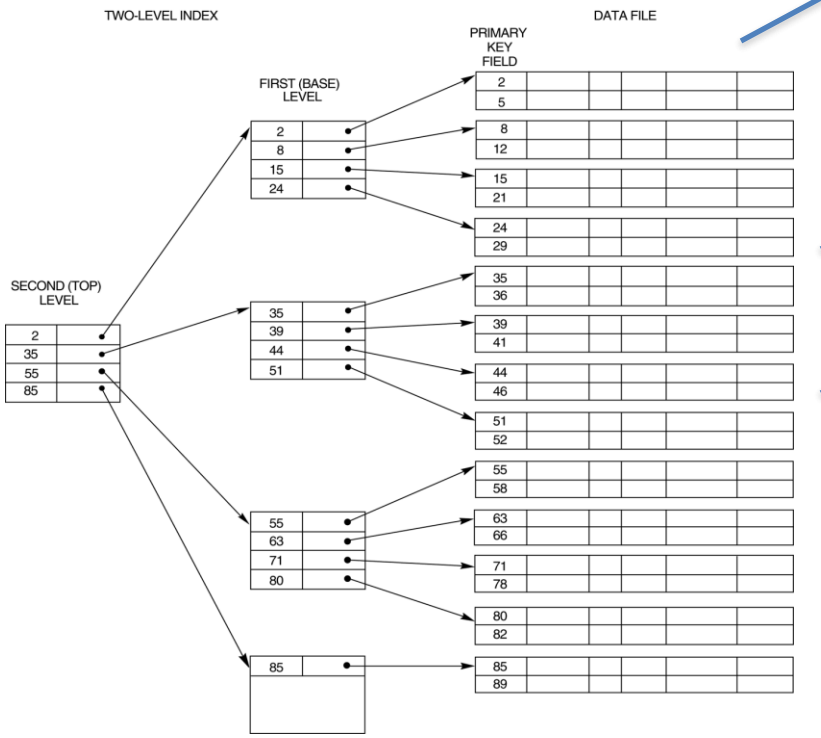
ID	Name	..	..	..	
1					
22					
300					
1000					
50					
2000					
..					
..					
..					
1000					
..					

Usage of an Index

```
Create Index on Student (Id);
```

Frontend

Build an index



Create index file

Create a specialized data structure

Whenever the table changes, the index needs to change

Backend

Transactions

Frontend

- Enroll "Mary Johnson" in "CSE444":

```
BEGIN TRANSACTION;

INSERT INTO Takes
    SELECT Students.SSN, Courses.CID
    FROM Students, Courses
    WHERE Students.name = 'Mary Johnson' and
                Courses.name = 'CSE444'

-- More updates here....

IF everything-went-OK
     THEN COMMIT;
ELSE ROLLBACK
```

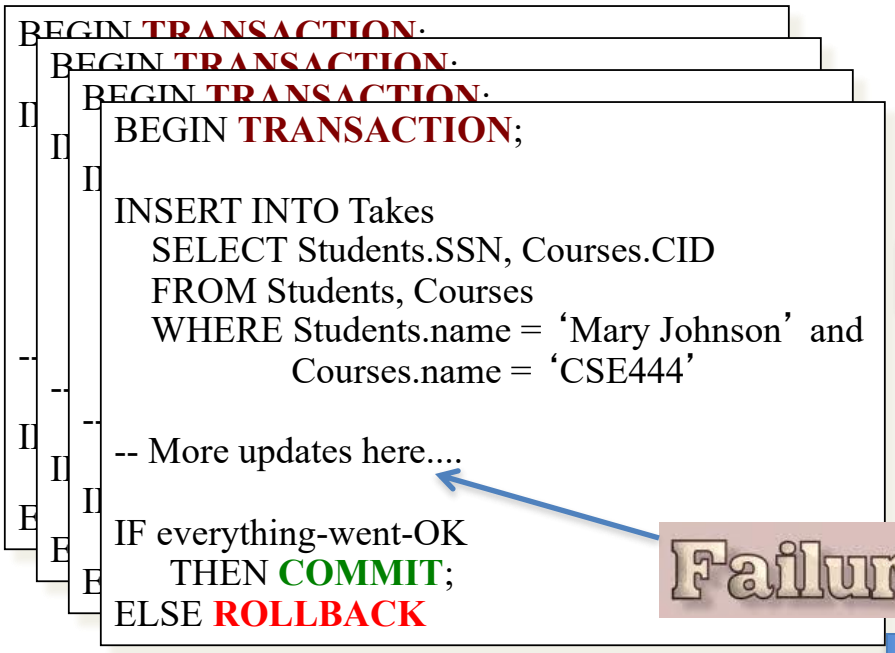
If system crashes, the transaction is still either committed or aborted

Transactions

- A *transaction* = sequence of statements that either all succeed, or all fail
- Basic unit of processing
- **Transactions have the ACID properties:**
  - A = atomicity
  - C = consistency
  - I = independence (Isolation)
  - D = durability

Transactions

Backend



Failure

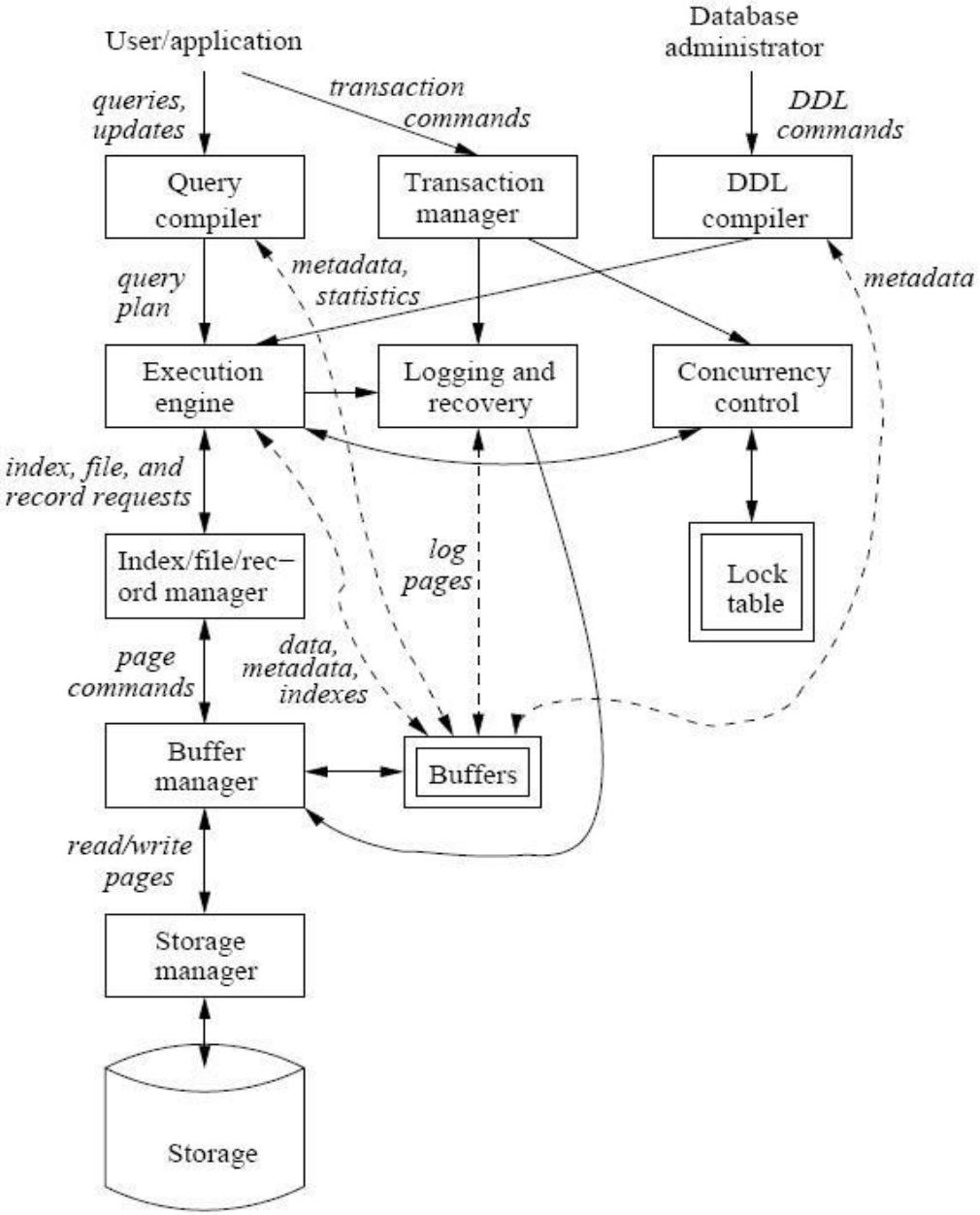
Many transactions at the same time (Concurrency Control)

Ensure the ACID properties

Logging and Recovery Control

DBMS Backend Components

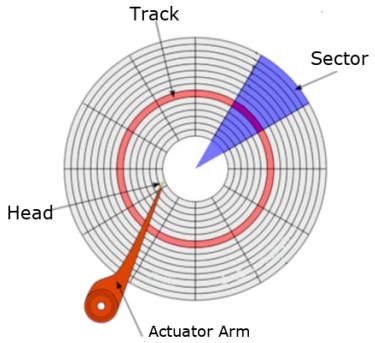
- We will cover several of these components

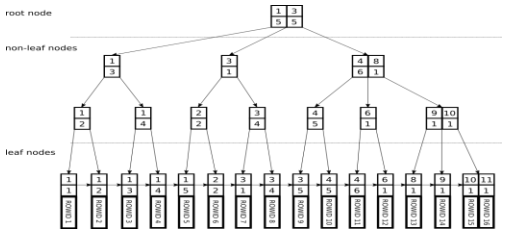


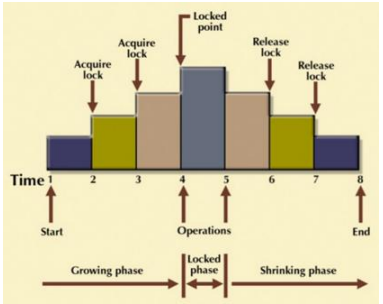
Database management system components

Topics To Be Covered…

- **File & System Structure**
  Records in blocks, dictionary, buffer management,…
- **Indexing & Hashing**
  B-Trees, hashing,…
- **Query Processing**
  Query costs, join strategies,…

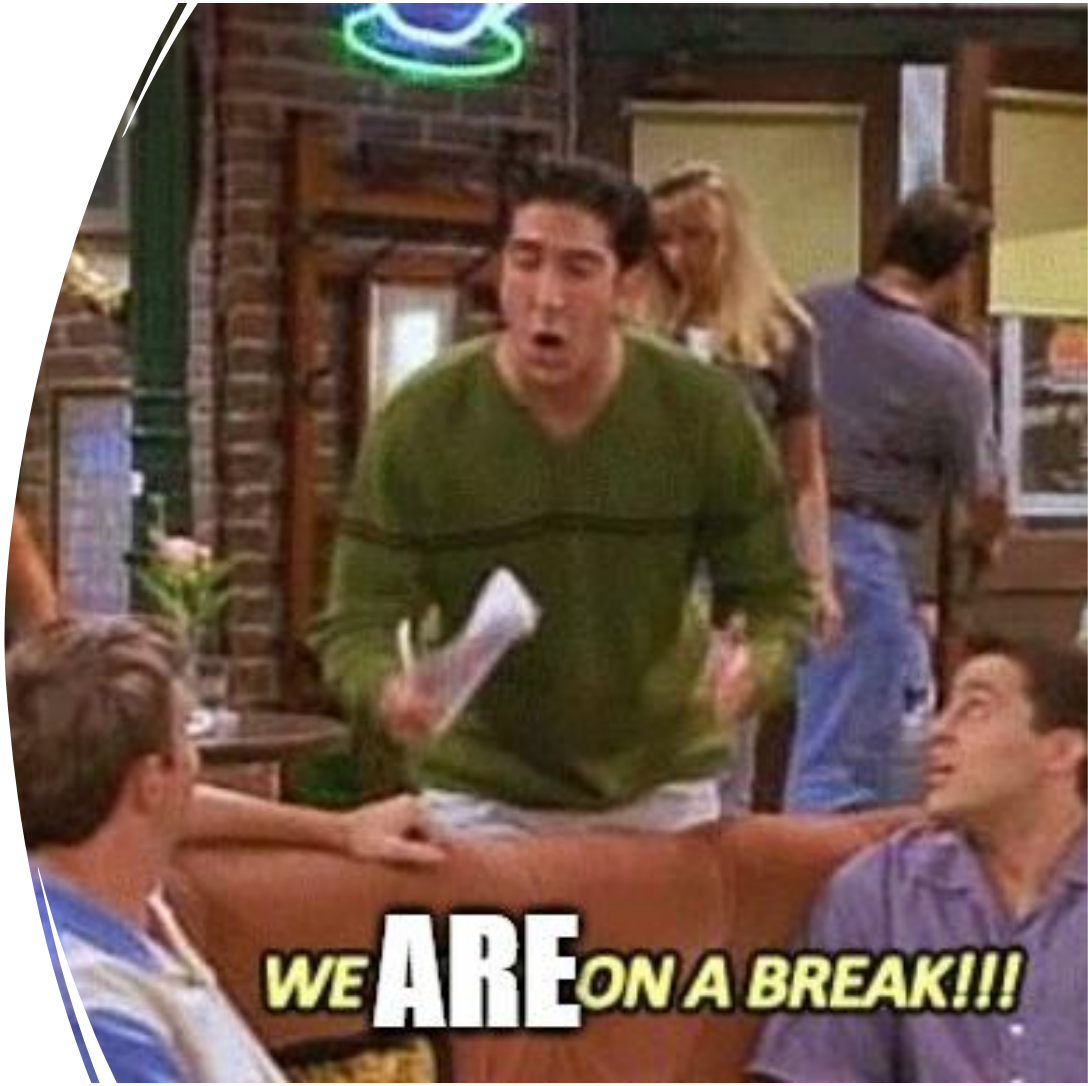






Break and “Quiz 0”

- Now we will take a few minutes break
- First, please login to Canvas and complete Quiz 0



Learning Goals

- Explain advantages of letting DBMS be in control of data (rather than OS)
- Identify purposes and characteristics of each layer in the storage hierarchy
- Identify magnetic disk components
- Explain how data is stored in disk
- Describe key costs involved in disk I/O & relationship to disk components
  - Explain advantages of Sequential I/O vs. Random I/O

Storage in DBMSs

How to organize & store data?

- **Design decisions:**
  - What representations and data structures best support efficient manipulations of this data?

- To understand why the DBMSs applies specific strategies
  - ***Must first understand how disks work***

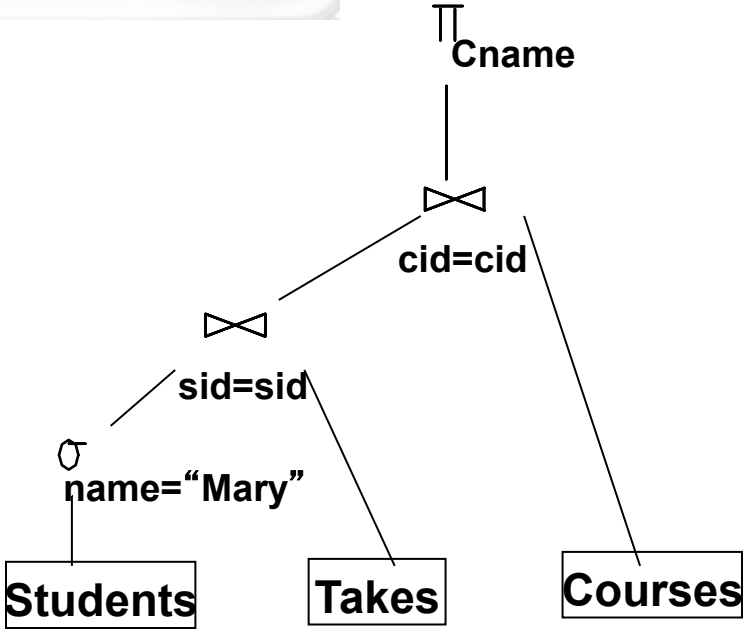
Disks and Files

- DBMS stores information on ("hard") disks.
- Main memory is only for processing

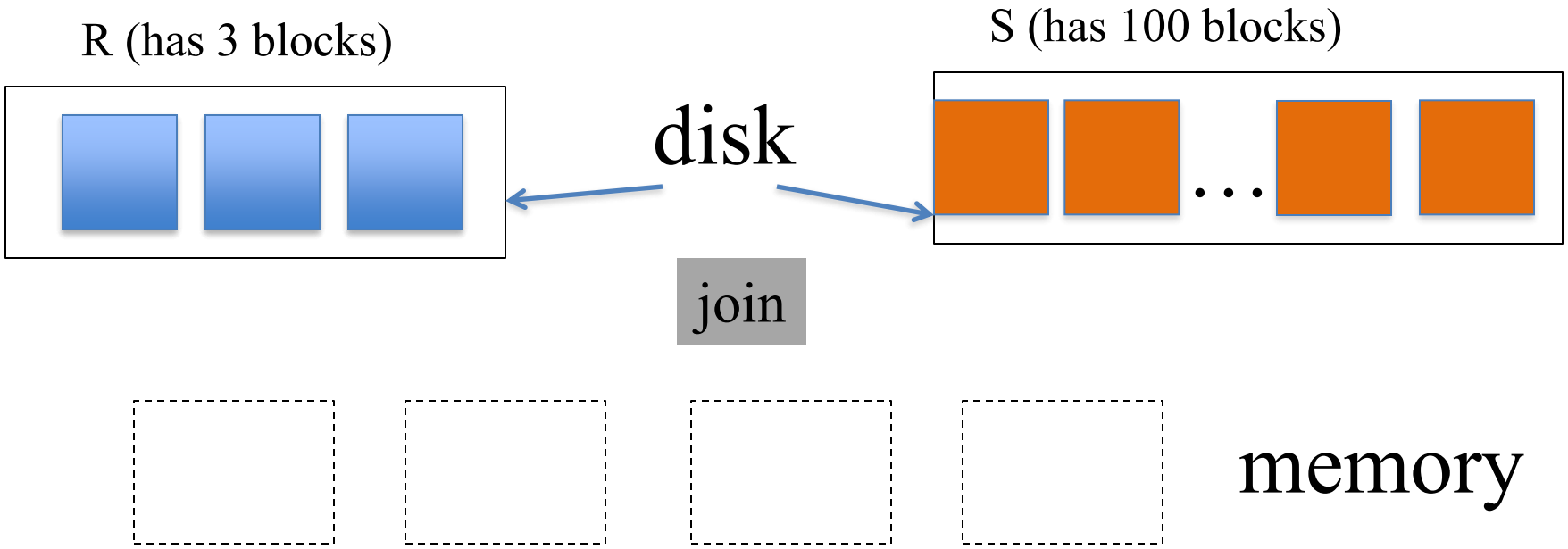
- This has major implications for DBMS design!
  - READ: transfer data from disk to main memory (RAM).
  - WRITE: transfer data from RAM to disk.
  - ***Both are high-cost operations (I/O), relative to in-memory operations…***

DBMS vs. OS? Who's in Control

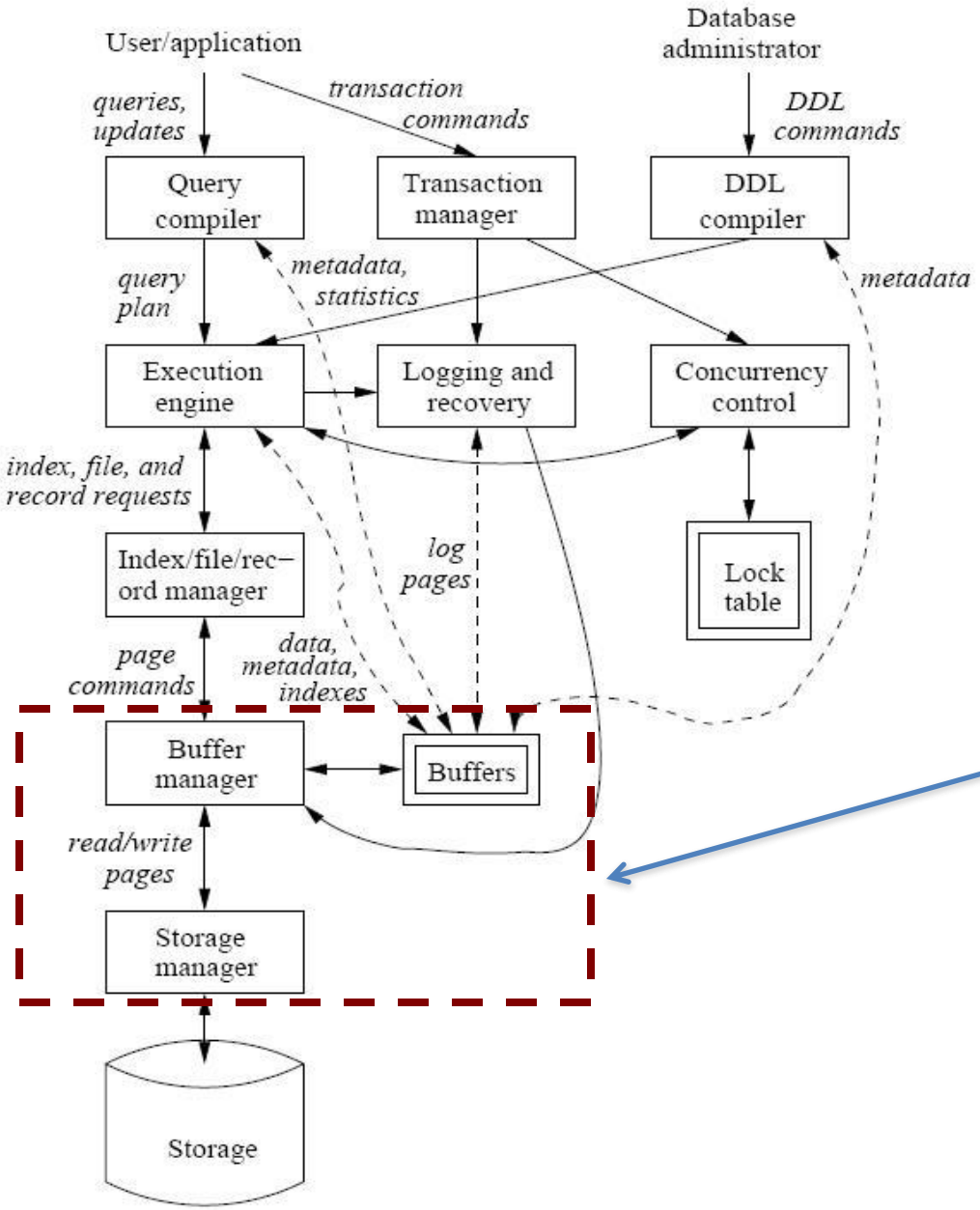
- **DBMS is in control of managing its data**
  - It knows more about structure
  - It knows more about access pattern



Example: Why DBMS should be in control

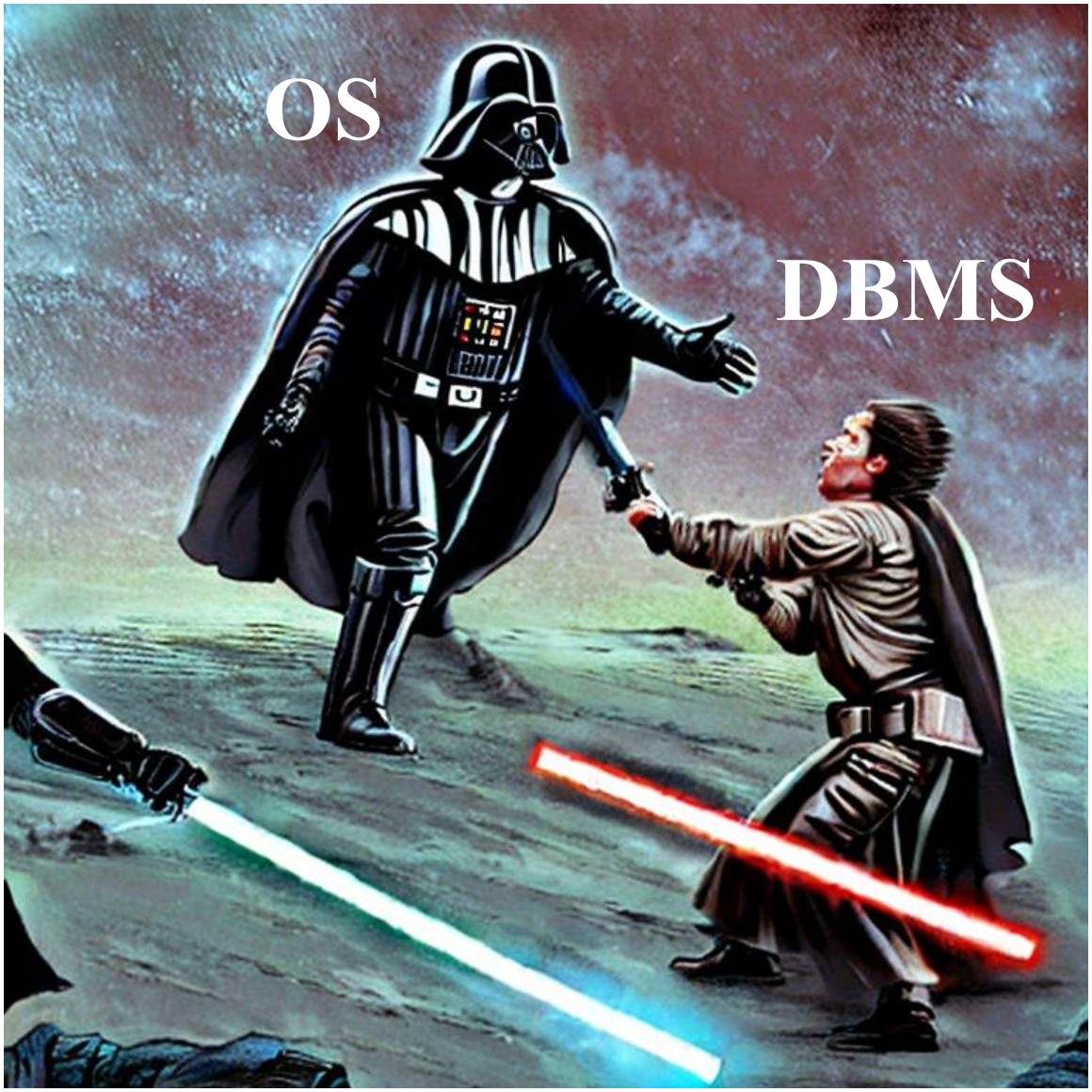


- OS does not know this logic, i.e., DB is trying to do join
- OS may take out one or more of R's blocks if it needs to free some memory... *This is something the DB will NOT do*



Database management system components

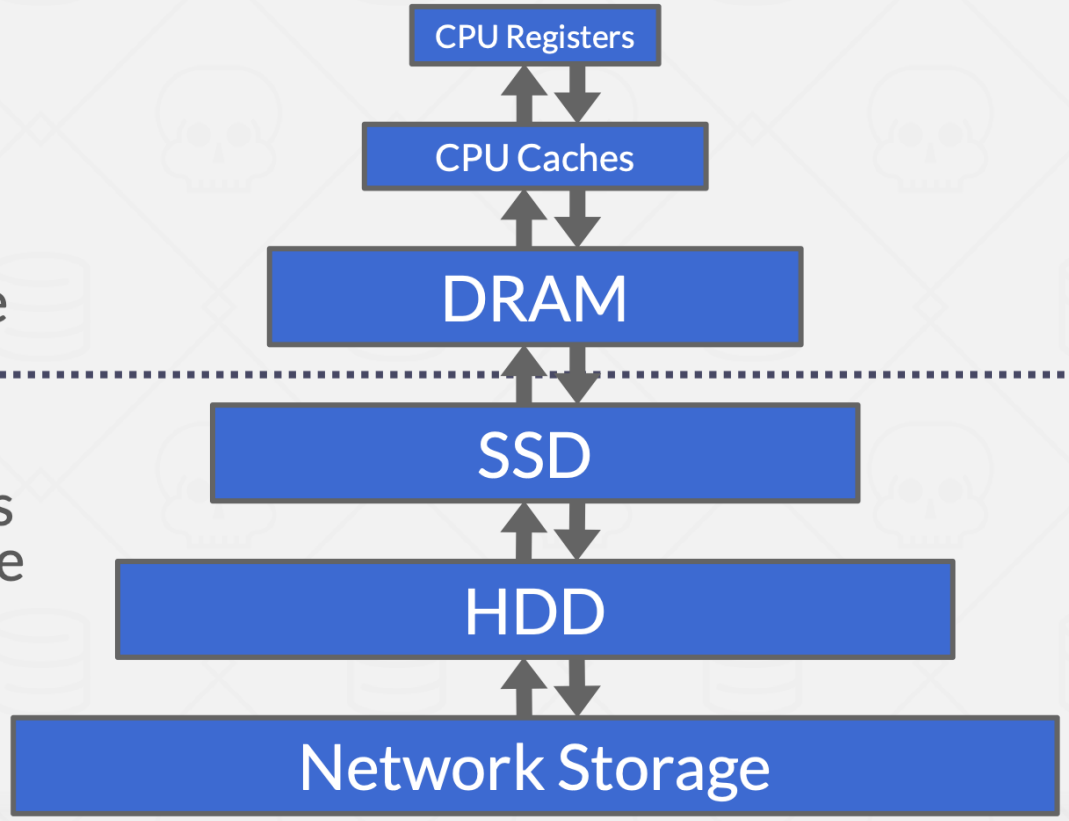
That is why DBMS has Storage Manager & Buffer Manager



Stable Diffusion 2.1 prompt: Vader fighting Jedi

Understanding Disks

STORAGE HIERARCHY



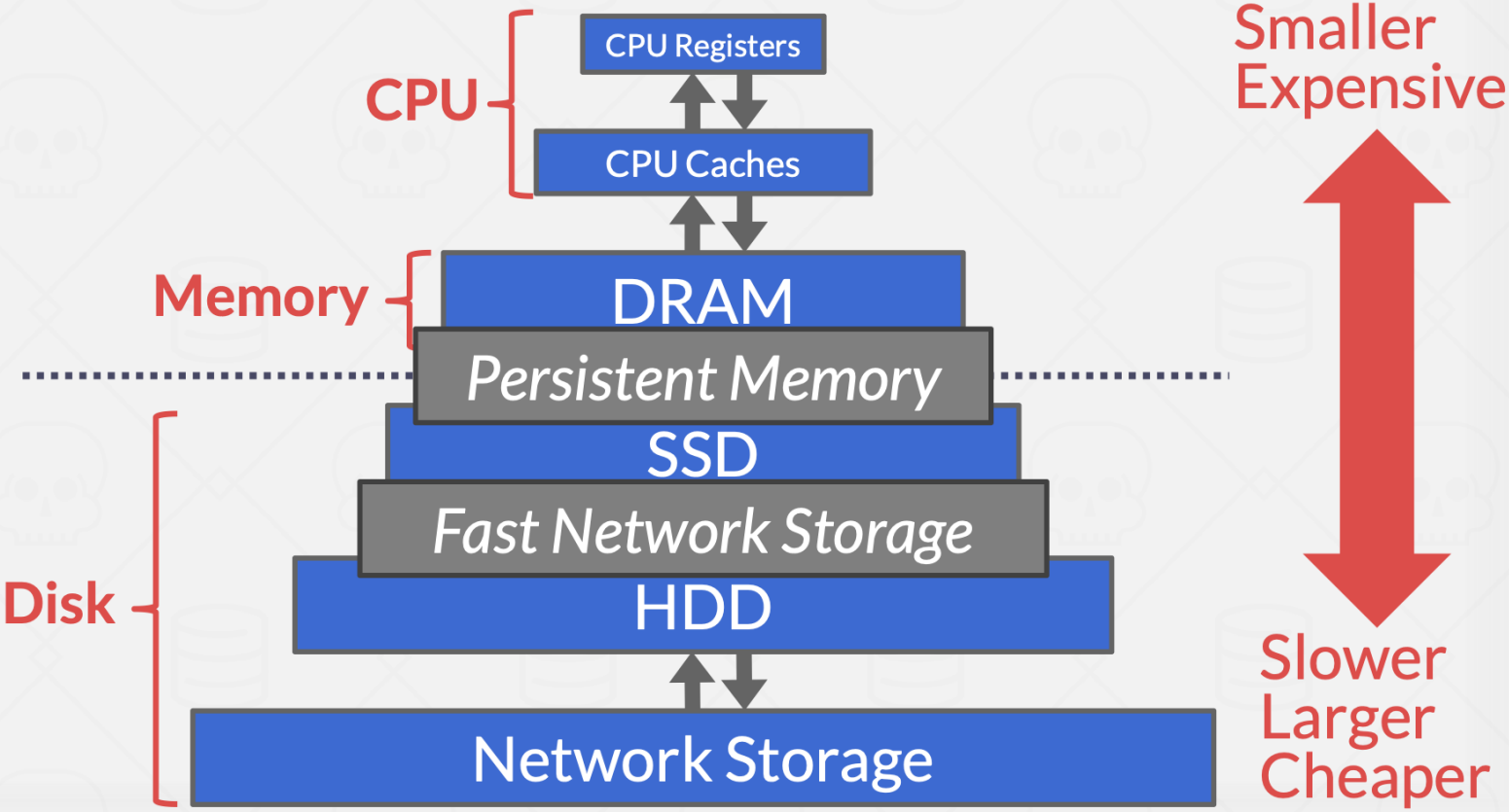
Volatile
Random Access
Byte-Addressable

Non-Volatile
Sequential Access
Block-Addressable

Faster
Smaller
Expensive

Slower
Larger
Cheaper

STORAGE HIERARCHY



Access Times

Latency Numbers Every Programmer Should Know

←1 sec

←4 sec

←100 sec

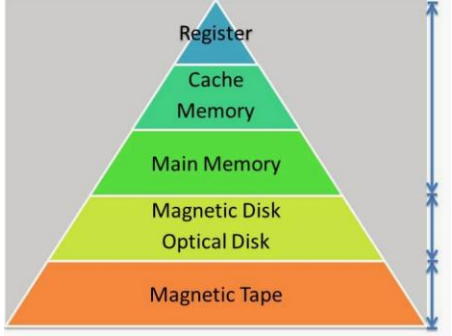
←4.4 hours

←3.3 weeks

←1.5 years

←31.7 years

Source



Storage Hierarchy

Fastest

Slowest

Cache (all levels)

Main Memory

Secondary Storage

Tertiary Storage

Avg. Size: 8kb-16MB

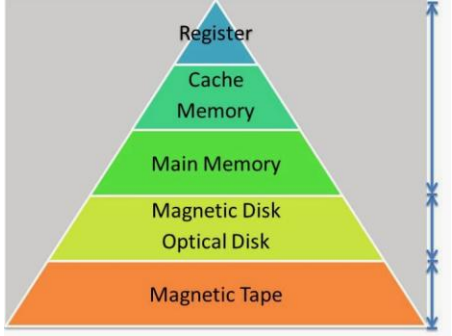
Read/Write Time: 1 to 10 ns

Random Access

Smallest of all memory, and also the most costly.

Usually on same chip as processor.

Easy to manage in Single Processor Environments, more complicated in Multiprocessor Systems.



Storage Hierarchy

Fastest

Slowest

- Cache (all levels)
- Main Memory
- Secondary Storage
- Tertiary Storage

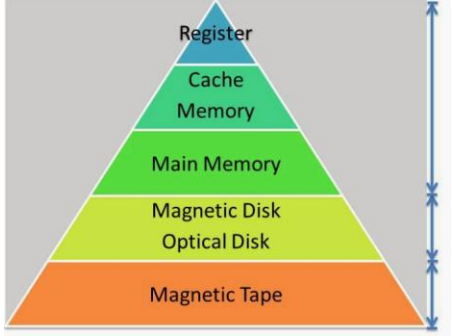
Avg. Size: 128 MB – 100s GB

Read/Write Time: 10^{-7} to 10^{-8} seconds.

Random Access

Becoming more affordable.

Volatile



Storage Hierarchy

Fastest

Slowest

Cache (all levels)

Main Memory

Secondary Storage

Tertiary Storage

Avg. Size: 100s GB- few TBs

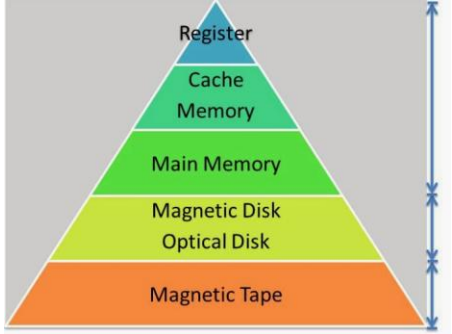
Read/Write Time: 10^{-2} seconds

NOT Random Access

Extremely Affordable: $0.025/GB!!!

Can be used for File System, Virtual Memory, or for raw data access.

Blocking (need buffering)



Storage Hierarchy

Fastest

Slowest

- Cache (all levels)
- Main Memory
- Secondary Storage
- Tertiary Storage

Avg. Size: 100s of Terabytes

Read Time: 50-60 seconds
Write Time: 10^{-2} seconds

NOT Random Access, or even remotely close

Extremely Affordable: pennies/GB!!!

Not efficient for any real-time database purposes, could be used in an offline processing environment

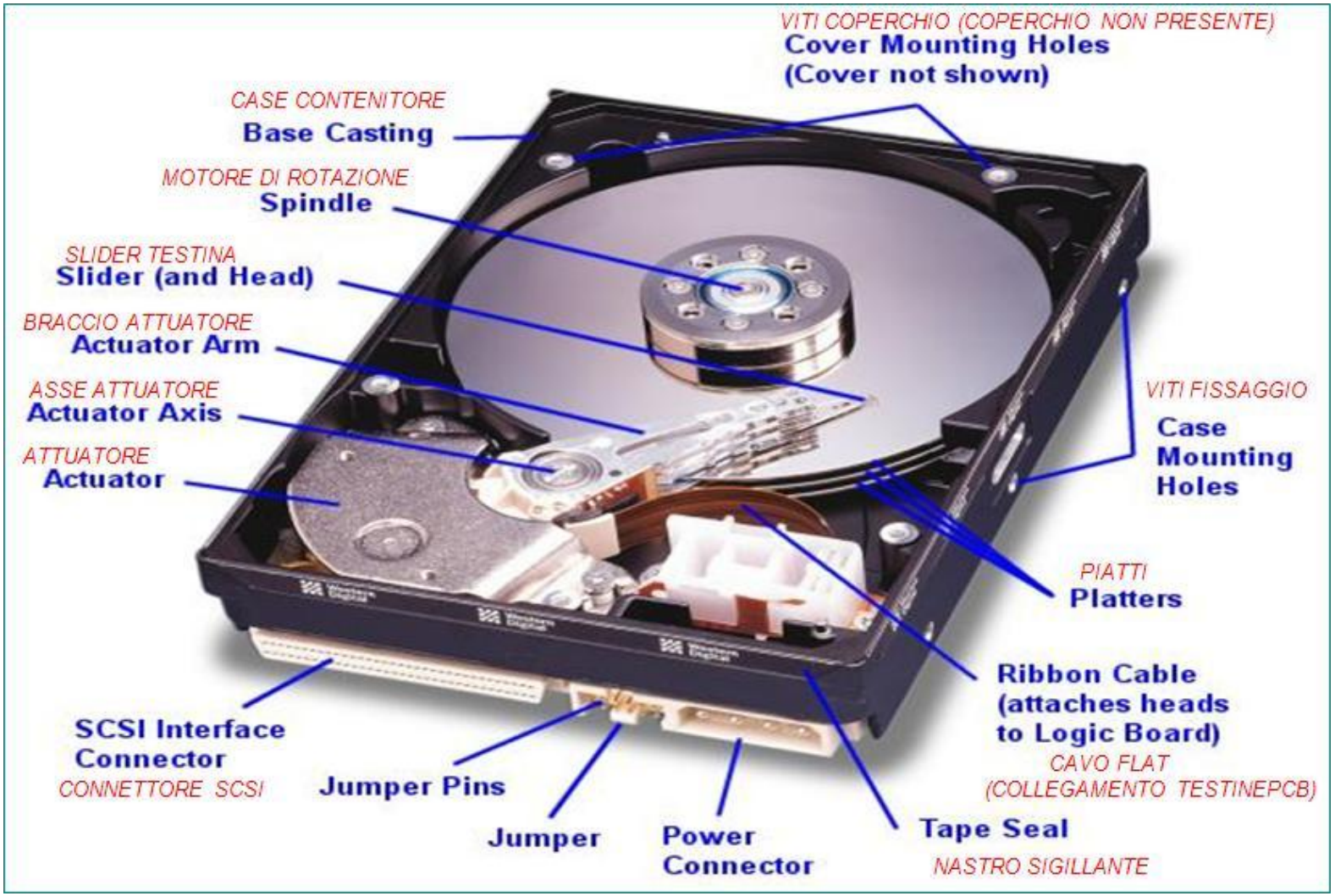
Why Not Store Everything in Main Memory?

- ***Costs too much.*** $100 will buy you either 16GB of RAM or 4TB of HDD today.

- ***Main memory is volatile.*** We want data to be saved between runs. (Obviously!)

- **Typical hierarchy:**
  - Main memory (RAM) ← **Processing**
  - Disks (secondary storage) ← Persistent Storage
  - Tapes, DVDs & Cloud ← **Archival**

Disks

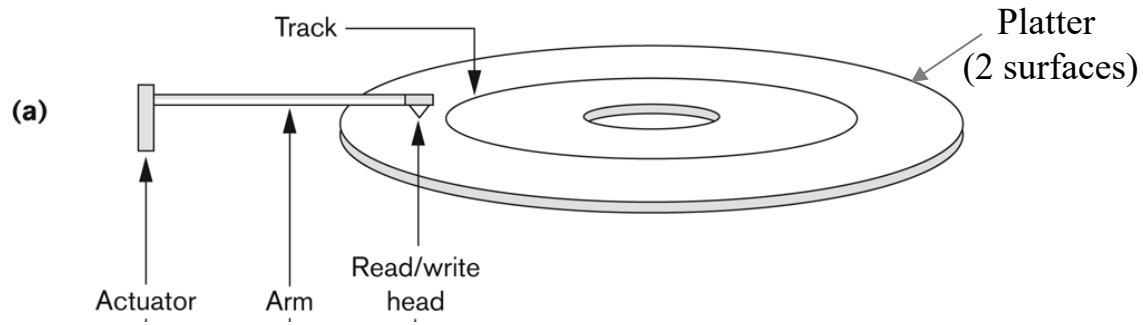

VITI COPERCHIO (COPERCHIO NON PRESENTE)
Cover Mounting Holes
(Cover not shown)
CASE CONTENITORE
Base Casting
MOTORE DI ROTAZIONE
Spindle
SLIDER TESTINA
Slider (and Head)
BRACCIO ATTUATORE
Actuator Arm
ASSE ATTUATORE
Actuator Axis
ATTUATORE
Actuator
VITI FISSAGGIO
Case Mounting Holes
PIATTI
Platters
Ribbon Cable (attaches heads to Logic Board)
CAVO FLAT (COLLEGAMENTO TESTINEPCB)
SCSI Interface Connector
CONNETTORE SCSI
Jumper Pins
Jumper
Power Connector
Tape Seal
NASTRO SIGILLANTE

Disks: Some Facts

- Data is stored and retrieved in units called *disk blocks* (sometimes also referred to as "pages")
  - Disk block typically between 4K to 8K
  - "K" (Kilobyte) → 1024 bytes
- Movement to main-memory
  - Must read or write one block at a time

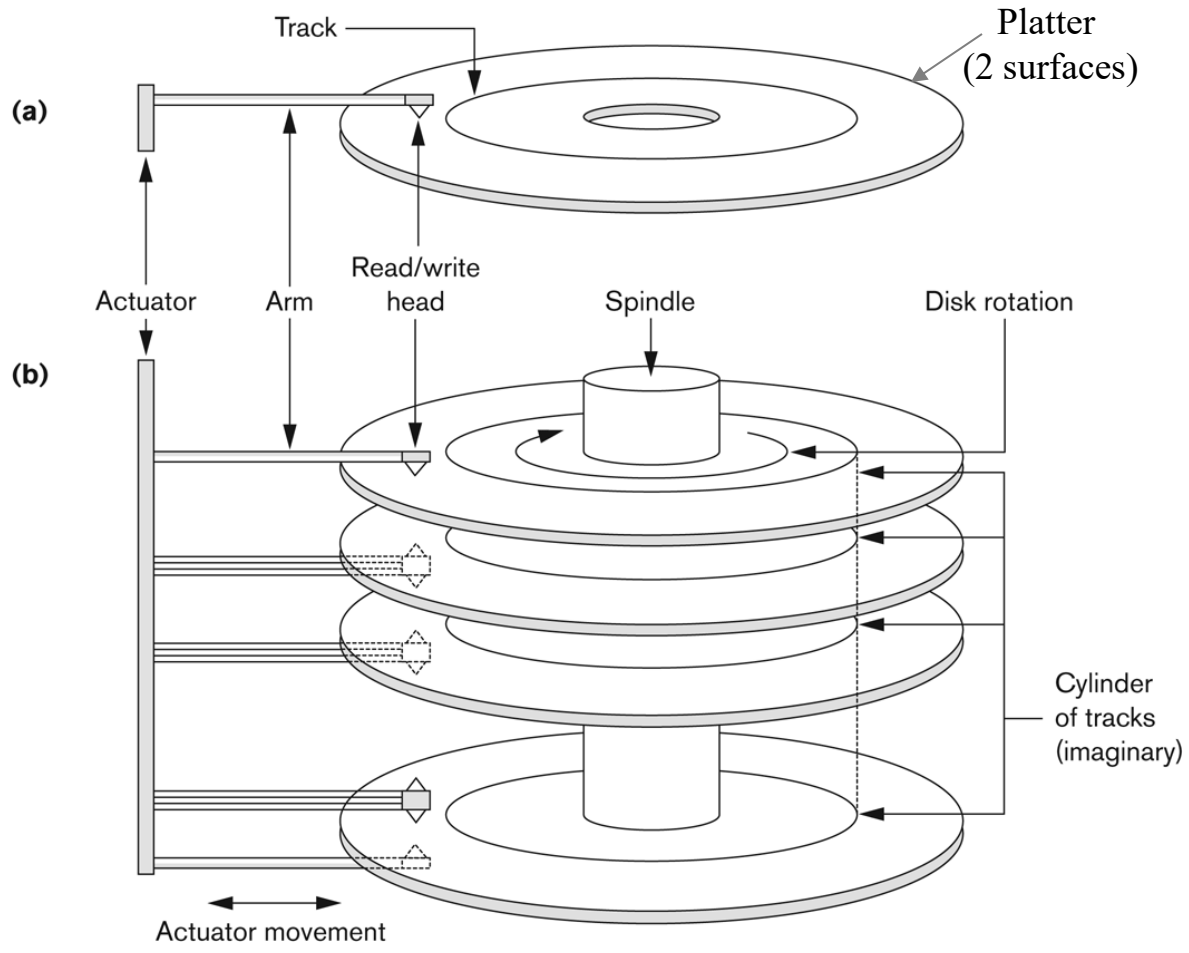
Disk Components

Figure 13.1
(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.

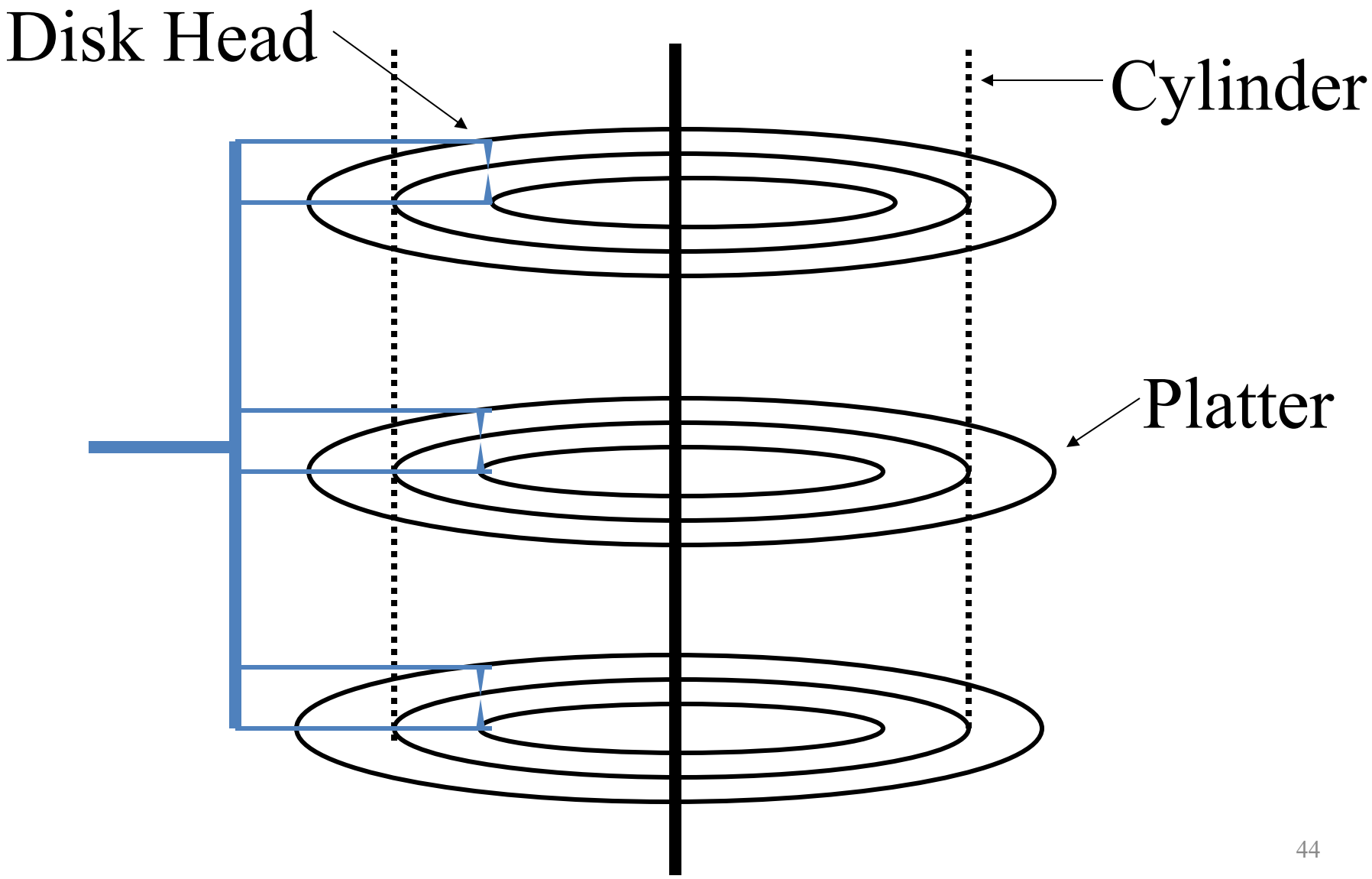


Disk Components

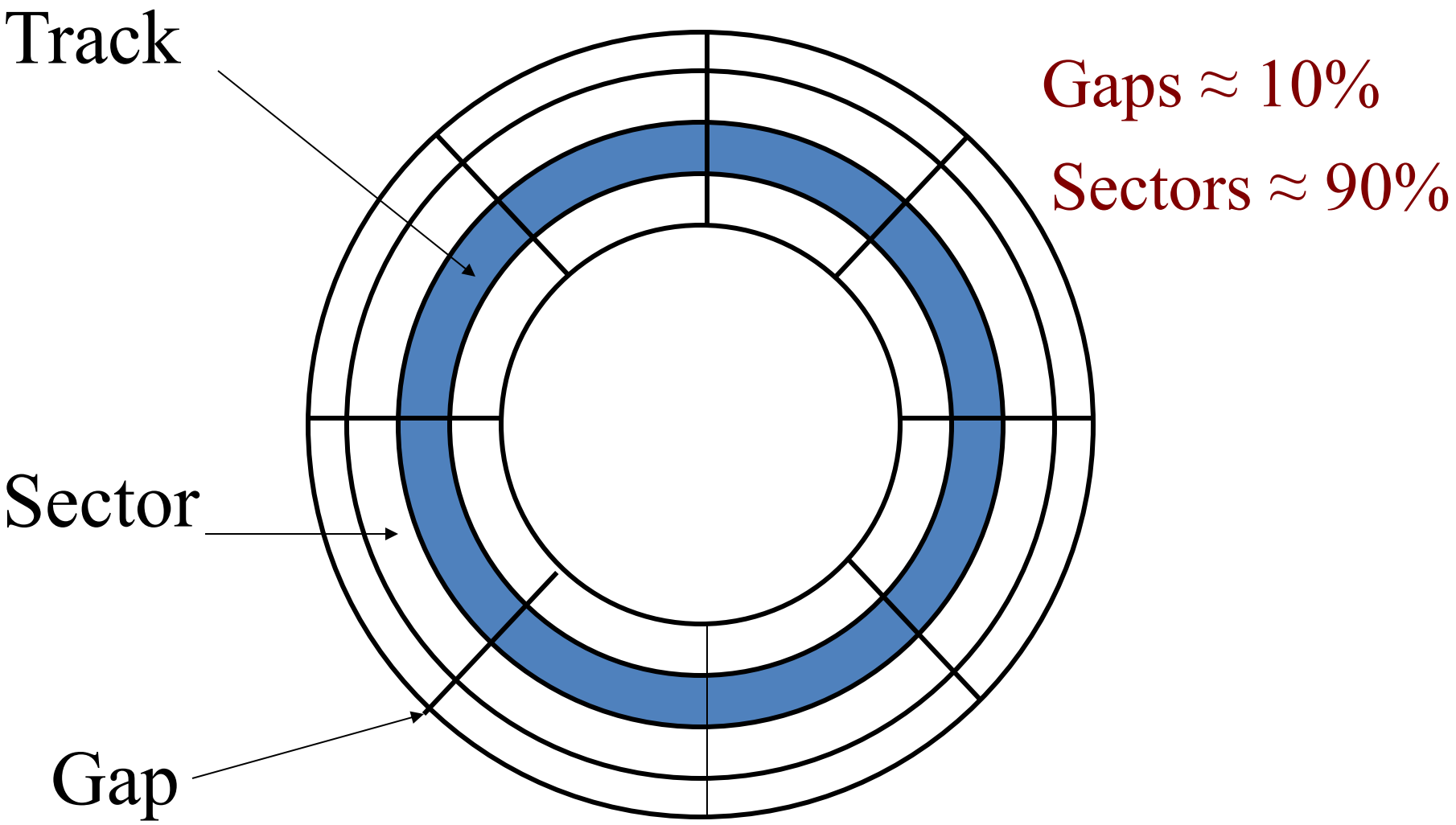
Figure 13.1
(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.



Virtual Cylinder



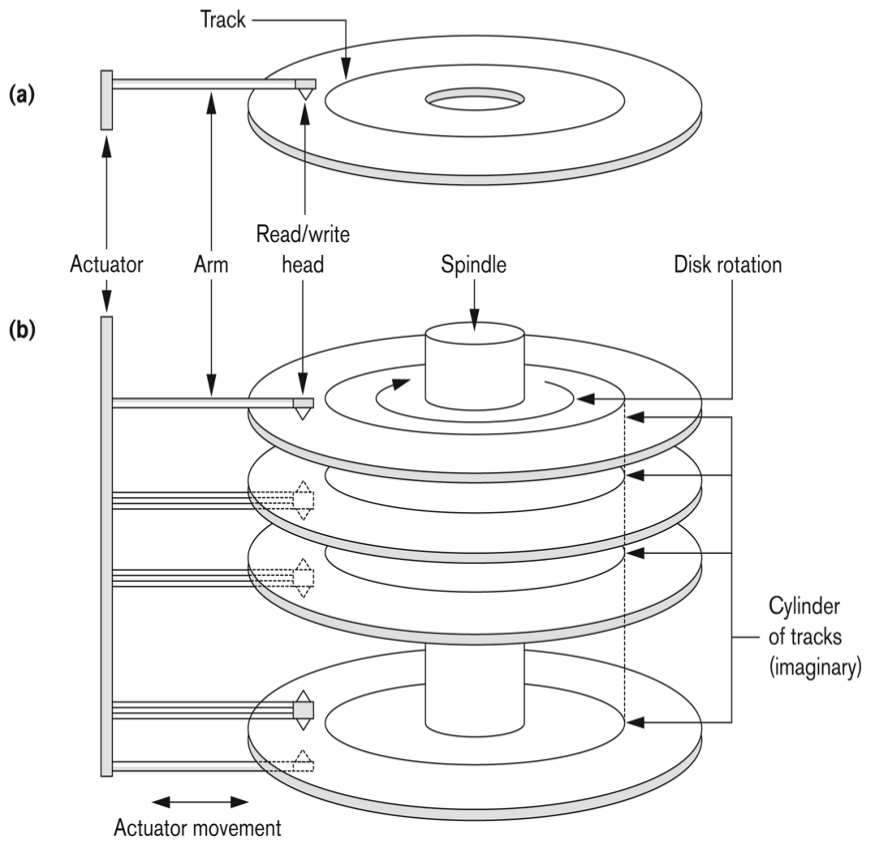
Tracks divided into Sectors



Movements

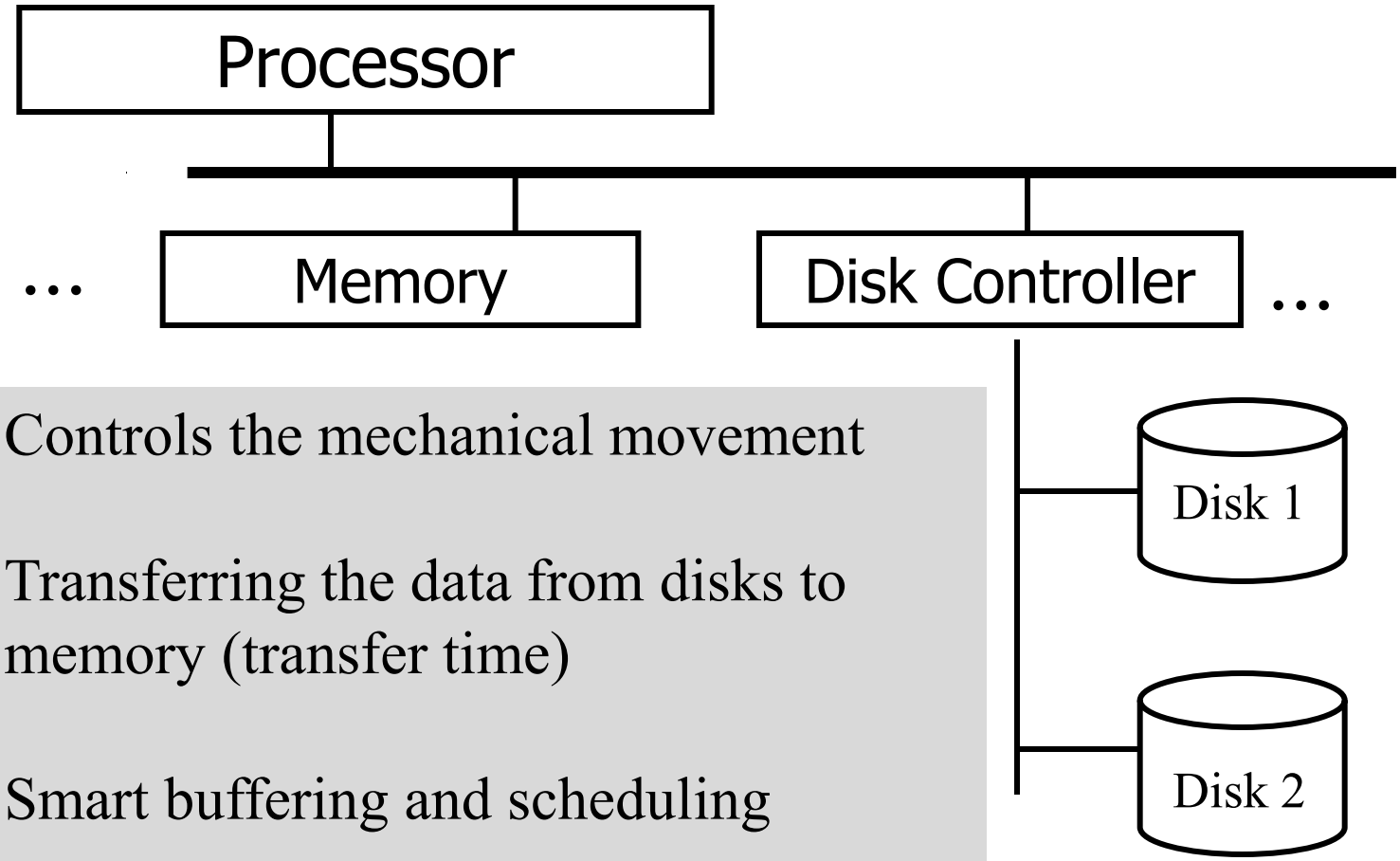
- Arm moves in-out
  - Called ***seek time***
  - Mechanical
- Platter rotates
  - Called ***latency (or rotation) time***
  - Mechanical

Figure 13.1
(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.



https://youtu.be/n6uPALWAyxc?si=KNkarxqjb3e0nOrP

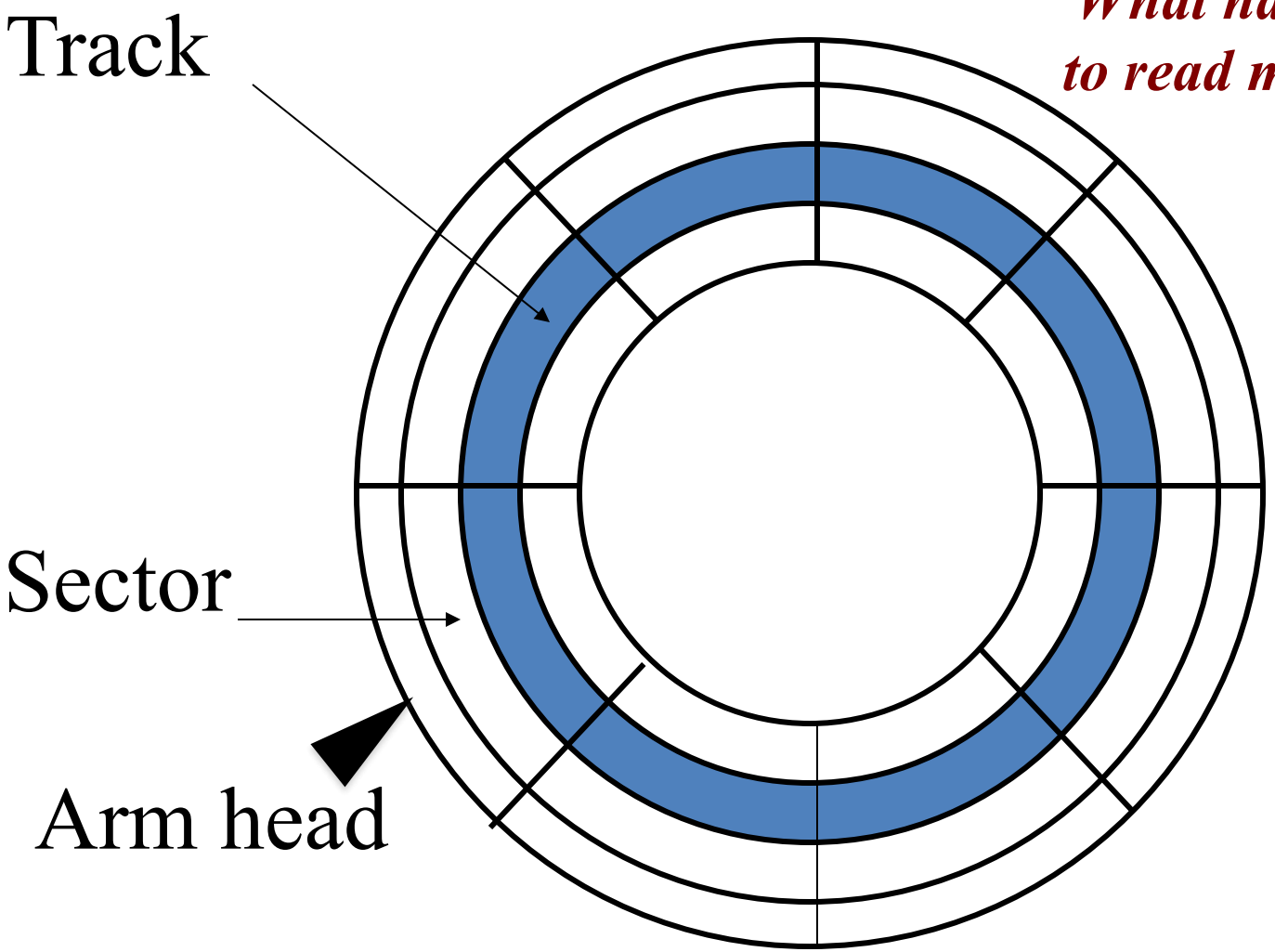
Disk Controller



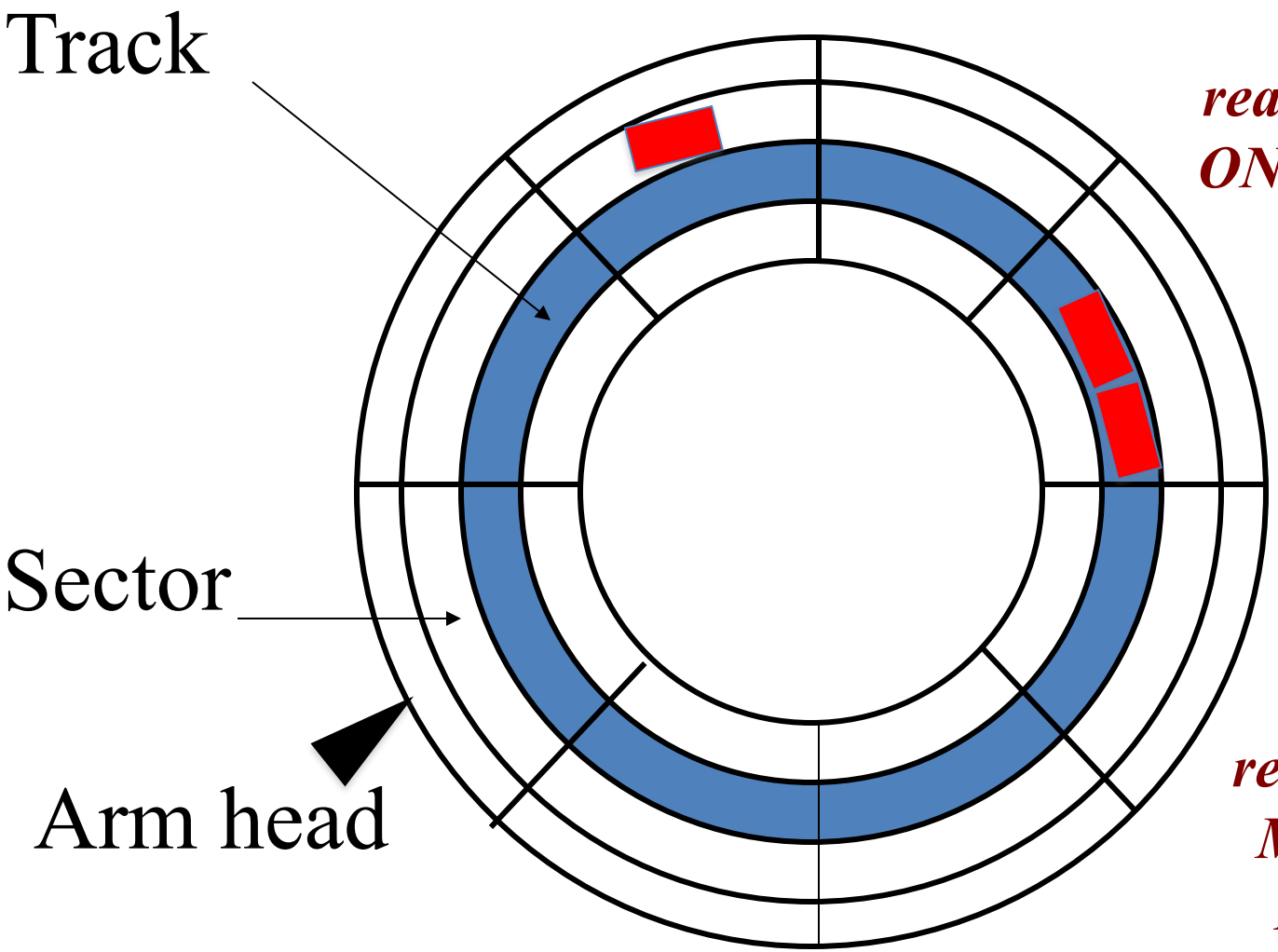
1. Controls the mechanical movement
2. Transferring the data from disks to memory (transfer time)
3. Smart buffering and scheduling

Sequential vs. Random I/O

What happens now if I want to read more than one block?



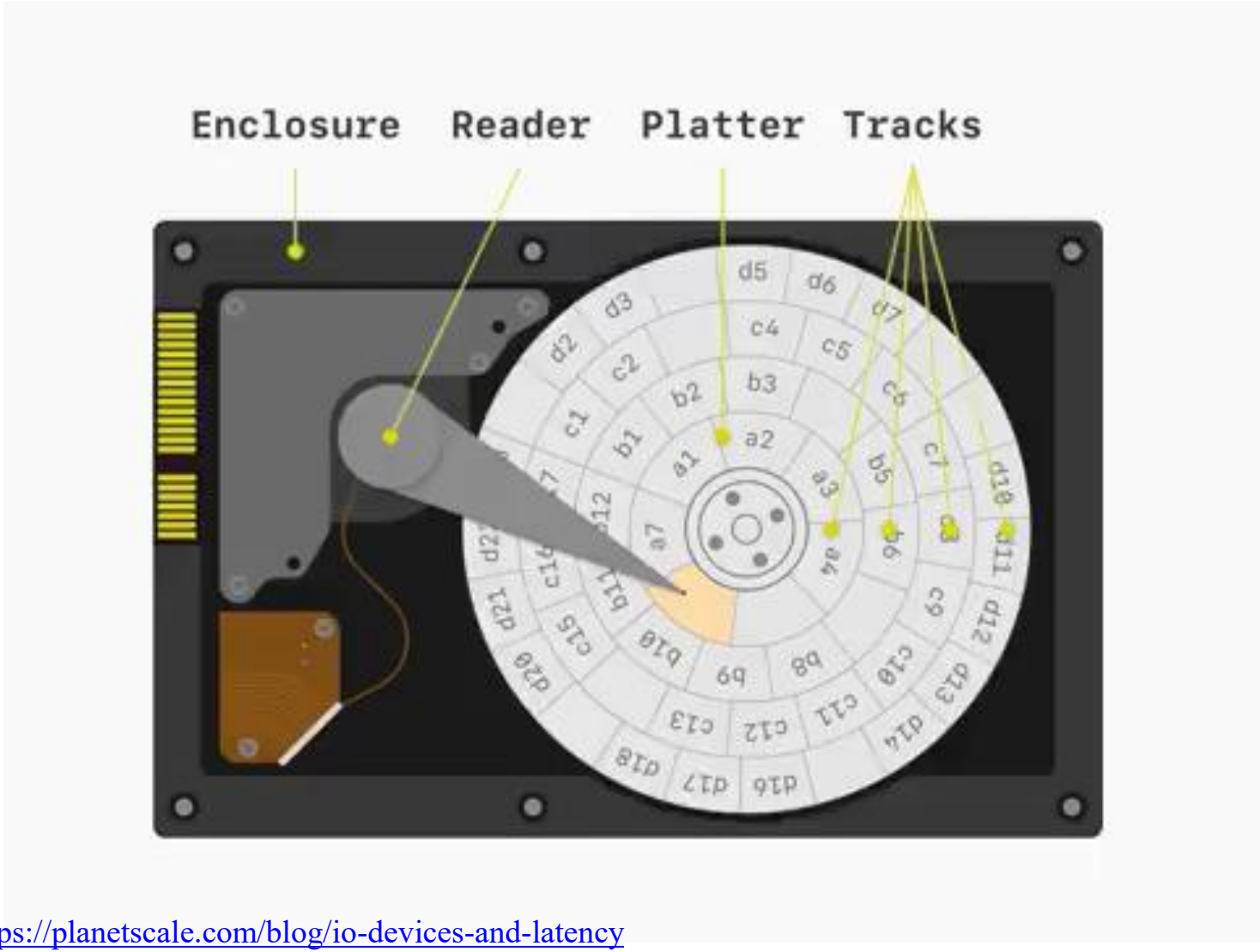
Sequential vs. Random I/O



Sequential reads/writes involve ONE seek and ONE latency cost

Random reads/writes involve Multiple seek and Multiple latency costs

Recap: Tracks divided into Sectors



Resource: https://planetscale.com/blog/io-devices-and-latency

How long does it take?

- Read two random blocks
  - 2 seek times
  - 2 latency times
  - 2 transfer times

Disk Delay{seek & rotation} +
Transfer Time

- Read two blocks in a sequence
  - 1 seek time
  - 1 latency time
  - 2 transfer times

2nd Disk Delay{seek & rotation}
Transfer Time

Summary

- Disk is for permanently storing data
- For any processing, data must be in memory
- Transfer between disk and memory must be in the unit of "disk blocks" (4kbytes to 8kbytes)
- Disk I/O operations are expensive because of the mechanical movement
  - Seek time
  - Latency (rotation) time
- One I/O operation is equivalent to 10^5 main memory operations
- Disk Anatomy (platters, surfaces, tracks, sectors, cylinders)

Take-home message